

Plateforme Converged AI

Converged AI est une plateforme open-source pour les entreprises de production : services d'impression 3D, ateliers CNC, laboratoires R&D et réseaux de petits ateliers.

Son rôle est de prendre en charge tout ce qui entoure la production : demandes, fichiers, estimations, files d'attente, statuts de commande, communication client, équipements, paiements, livraison et ventes récurrentes. Les machines continuent de fabriquer les pièces, l'équipe continue de prendre les décisions techniques, et Converged relie tout cela dans un système pilotable.

La plateforme comprend **17 solutions** regroupées en quatre domaines : commandes et clients, production et stocks, argent et rentabilité, équipe et responsabilité. Ce ne sont pas des « modules pour faire des modules », mais des scénarios prêts à l'emploi pour des problèmes concrets d'atelier.

Ce que fait Converged

Ce que fait Converged

Une entreprise de production a deux couches de travail. La première consiste à fabriquer la pièce, assembler le produit et terminer la commande. La seconde regroupe tout ce qui doit se passer avant, pendant et après la production : recevoir une demande, ne pas perdre le fichier, valider le prix, placer le travail dans une file, prévenir le client, comprendre la charge des machines, voir les retards et encaisser le paiement.

Converged couvre cette deuxième couche. C'est un circuit numérique de contrôle qui relie le site web, les demandes clients, les tâches internes, les équipements, les stocks, les paiements, les notifications et l'analytics. Le dirigeant ne voit pas une collection d'outils séparés, mais une image vivante de l'entreprise : ce qui est arrivé, ce qui est accepté, ce qui est en file d'attente, ce qui est prêt, où le retard menace et où la marge disparaît.

La plateforme est particulièrement utile lorsque la production fonctionne déjà, mais que la gestion repose sur des chats, des tableurs, la mémoire des employés et du contrôle manuel. Converged n'impose pas immédiatement un ERP ou un MES lourd. Il commence par des processus concrets et construit progressivement un système unifié autour d'eux.

Dans un scénario typique, le client envoie une demande via le site, un formulaire, une messagerie ou un opérateur. Converged enregistre la demande, attache les fichiers, crée une commande, aide à estimer le prix, place le travail en file, suit l'exécution et tient le client informé. Un opérateur peut demander le statut dans l'interface ou via le chat IA, et le système répond à partir de données réelles, pas d'hypothèses.

Résultat : l'atelier dépend moins de la coordination manuelle. Les personnes se concentrent sur la production et les décisions qui exigent réellement de

l'expérience, tandis que la plateforme gère le routage, les délais, les messages répétitifs, la collecte des statuts et la préparation des actions.

Système de solutions

Système de solutions

Converged n'est pas vendu comme une plateforme vide où le client doit d'abord inventer l'architecture et assembler des modules. L'unité de valeur de base est une **solution** : un scénario de travail prêt à l'emploi qui ferme un problème métier clair.

La plateforme prévoit **17 solutions** regroupées en quatre domaines :

- **Commandes et clients** — demandes entrantes, vitrine de services, historique client, statuts, communication et ventes récurrentes.
- **Production et stocks** — charge des équipements, files d'attente, matériaux, contrôle qualité, pannes et expéditions.
- **Argent et rentabilité** — coûts, marge, paiements, créances, tarification et scénarios de croissance.
- **Équipe et responsabilité** — zones de responsabilité, équipes, standards, base de connaissances et onboarding.

Une solution dans Converged n'est pas seulement un écran d'interface. Elle inclut généralement un modèle de données, un workflow, des rôles, des notifications, des actions d'agent IA et des intégrations avec des équipements ou des services externes. Le dirigeant ne choisit pas une « fonctionnalité », mais un problème : accélérer le traitement des demandes, voir la file des machines, comprendre la rentabilité des commandes ou structurer les équipes.

La description détaillée de toutes les solutions doit vivre dans une section séparée. Dans la documentation produit, l'essentiel est le principe : Converged AI est la plateforme, et les solutions sont des scénarios applicatifs qui fonctionnent à l'intérieur et couvrent progressivement différentes zones de l'entreprise de production.

Équipements et atelier

Équipements et atelier

Converged ne remplace pas le firmware des machines et ne tente pas de piloter la production à l'aveugle. Il se connecte au-dessus des équipements, lit la télémétrie, relie l'état des machines aux commandes et montre ce qui se passe réellement dans l'atelier.

La plateforme est conçue pour différents types d'équipements : imprimantes 3D Bambu Lab, Marlin et Klipper, machines CNC, cellules robotisées et adaptateurs spécialisés. Lorsque c'est possible, Converged reçoit les statuts, erreurs, progrès d'exécution, températures, files de tâches et autres données techniques. Lorsque le contrôle direct est risqué ou indisponible, le système reste une couche d'observation et de coordination.

Pour le dirigeant, cela signifie une chose simple : les équipements ne sont plus une série de fenêtres séparées. On voit quelles machines sont occupées, où il y a de l'inactivité, ce qui prend du retard, quelle commande est liée à une opération précise et où un opérateur doit intervenir. À mesure que le parc grandit, cette visibilité devient plus importante que le passage manuel entre les interfaces de chaque imprimante ou machine.

Converged relie aussi les équipements au processus métier. Une tâche n'est pas simplement « en impression » ou « en fraisage » : elle existe dans le contexte d'une commande, d'un délai, d'un client, d'un matériau, d'un paiement et de l'étape suivante. L'atelier devient une partie du système commun, pas une île séparée.

Processus

Processus

Le principal problème d'un atelier qui grandit est rarement l'absence d'un bouton supplémentaire. Le plus souvent, les processus vivent dans la tête des personnes : qui doit répondre au client, quand calculer le prix, qui vérifie le fichier, quand lancer la production, qui prévenir en cas de retard et que faire après l'expédition.

Dans Converged, ces chaînes sont décrites comme des workflows. Un processus typique peut aller de la demande à l'estimation, l'approbation, la mise en file, la production, le contrôle qualité, le paiement, la livraison et les notifications. L'utilisateur ne construit généralement pas un graphe à partir de zéro : les scénarios prêts à l'emploi sont livrés avec les solutions, et la configuration se limite aux règles, rôles, délais, intégrations et notifications.

Techniquement, l'exécution est déplacée dans la couche Runtime. Elle lance les workflows, tâches cron, étapes d'intégration et logique métier tout en restant stateless : les données persistantes restent dans les microservices, et Runtime exécute les chaînes. Ainsi, la logique métier n'est pas dispersée dans des dizaines de services et il existe un endroit clair où vivent les règles de processus.

Pour les déploiements complexes, les workflows peuvent être étendus. Un développeur décrit les scénarios comme des classes TypeScript typées, et les agents IA peuvent lancer des actions autorisées dans ces scénarios. Mais pour un utilisateur ordinaire, l'objectif est différent : ne pas construire un éditeur, mais activer un processus prêt et obtenir un résultat maîtrisé.

Couche IA

Couche IA

L'IA dans Converged n'est pas un chat séparé ajouté pour l'apparence. C'est une couche de contrôle au-dessus des données, de l'interface et des processus.

L'utilisateur peut demander ce qui se passe avec une commande, préparer une réponse client, trouver un retard, lancer un workflow autorisé ou obtenir un résumé de la charge des machines.

Le modèle reçoit le contexte de la plateforme : demandes, statuts, fichiers, télémétrie, historique client, droits d'accès et tâches en cours. La réponse s'appuie donc sur les données de l'entreprise, pas sur des raisonnements génériques. Si une action est nécessaire, l'IA ne contourne pas le système : elle appelle des fonctions, microservices ou workflows autorisés via une couche contrôlée.

Les fournisseurs de modèles se connectent via des adaptateurs. Une même installation peut utiliser GPT, Claude, DeepSeek, Mistral, Gemini ou d'autres moteurs s'ils correspondent à la tâche et à la politique du client. Un modèle peut communiquer avec les clients, un autre analyser les cahiers des charges techniques, un troisième examiner les documents ou les statuts de production.

Le principe clé est le contrôle. L'IA a son propre profil d'accès, toutes les actions sont journalisées et les opérations critiques s'exécutent sous les mêmes droits et politiques que les actions humaines. Cela permet de confier la routine aux agents sans leur donner un pouvoir incontrôlé sur la production.

Architecture

Architecture

Converged est conçu comme une plateforme modulaire, mais pas comme une collection chaotique de microservices. La séparation est simple : l'interface affiche les données et lance les actions, Runtime exécute les processus, les microservices possèdent les données, et les adaptateurs connectent les équipements et systèmes externes.

Utilisateur / client ↓ **UI et micro-frontends** ↓ **Runtime :**
workflows, cron, intégrations, actions IA ↓ **Microservices : API**
typées et données propres ↓ **Storage / Behemoth / fichiers / SQL /**
KV / métriques ↓ **Équipements, messageries, paiements et services**
externes

Les microservices restent volontairement fins. Chaque service est responsable de son domaine de données, de la validation et d'une API typée. Il ne doit pas connaître la logique interne des services voisins ni devenir un centre caché de processus métier. Cela réduit le couplage et simplifie la maintenance.

Toute la logique transversale est déplacée dans Runtime. Si le système doit accepter une commande, interroger plusieurs services, créer une tâche, envoyer une notification, attendre un événement et mettre à jour un statut, cela s'exécute dans un workflow. Runtime ne stocke pas d'état persistant lui-même : il écrit l'historique, les variables et les résultats via les services qui possèdent leur stockage.

Le stockage est construit autour de l'isolation. Au lieu d'une base commune, chaque domaine obtient ses propres limites de données : SQL, key-value, fichiers, données colonnes, index vectoriels ou relations de graphe quand c'est nécessaire. Cette approche aide à déplacer les workspaces, limiter les accès et éviter une base partagée où les données de clients différents se mélangent.

Le frontend est également modulaire. Le shell commun charge des micro-frontends indépendants via import map, ce qui permet à certaines zones de

l'interface d'évoluer sans reconstruire tout le produit. Pour l'utilisateur, cela reste un seul système ; pour le développement, ce sont des zones de responsabilité claires.

Déploiement

Déploiement

Converged couvre plusieurs scénarios d'installation : du petit atelier au déploiement de production dans l'infrastructure de l'entreprise. La plateforme de base se déploie sur **k3s**, une distribution Kubernetes légère adaptée aux appareils edge, serveurs locaux et environnements cloud.

Il existe deux profils principaux :

- **Mono** — UI, Runtime, microservices, storage et cache sont regroupés de façon compacte. Ce mode sert au développement, aux prototypes, aux démonstrations et aux petites installations où la simplicité de démarrage est prioritaire.
- **Multi** — UI, groupes Runtime, groupes de microservices par domaine, storage et cache sont séparés. C'est le profil production standard lorsque l'isolation, la mise à l'échelle et le contrôle précis de la charge sont nécessaires.

Les deux profils utilisent le même code. Seules la topologie des conteneurs et la configuration changent. Une entreprise peut commencer par une installation compacte puis déplacer le même système vers une infrastructure plus sérieuse sans réécrire le produit.

En self-hosted, le client contrôle l'installation, le réseau, les sauvegardes, les mises à jour et l'emplacement physique des données. Cela convient aux entreprises avec des exigences internes de sécurité ou la volonté de garder la production entièrement chez elles. La livraison cloud retire les tâches opérationnelles : la plateforme est déployée et mise à jour par l'équipe du service, et le client reçoit un environnement prêt.

Une option hybride est aussi possible : les données sensibles et les équipements restent localement, tandis que le cloud sert aux mises à jour, à l'accès externe, à

la coordination d'équipes distribuées ou à certaines fonctions IA. Le principe important est de ne pas enfermer le client dans un seul modèle de livraison.

Technologies

Technologies

La partie serveur de Converged est construite sur **Bun** et **Elysia**. Bun lance rapidement JavaScript et TypeScript, consomme la mémoire efficacement et convient aux déploiements edge compacts. Elysia sert de couche HTTP pour les plugins backend et les microservices.

Les contrats entre services sont décrits avec des types. NRPC relie les interfaces TypeScript aux implémentations et génère des packages clients, afin que frontend, Runtime et backend travaillent avec les mêmes contrats plutôt qu'avec des API textuelles dispersées.

Le stockage utilise un ensemble de stores légers selon les tâches : SQL, key-value, fichiers, données colonnes, index vectoriels et relations de graphe. La couche native Behemoth et les adaptateurs Zig couvrent les cas où le faible overhead, l'accès aux équipements, les sockets Unix ou le FFI sont importants.

Le frontend est une plateforme React avec micro-frontends. Le shell commun charge des modules UI séparés, et les scénarios produit peuvent évoluer indépendamment. C'est important pour une plateforme avec de nombreuses solutions : l'interface ne doit pas devenir un monolithe lourd.

L'orchestration et la livraison s'appuient sur k3s, Helm et des profils de configuration. Le même ensemble de composants peut être assemblé en profil mono compact ou séparé en groupes pour la production.

Performance

Performance

Converged est conçu pour des sites de production qui ne disposent pas toujours d'un grand parc serveur. Le système évite donc le poids inutile : Bun réduit l'overhead des processus backend, Runtime reste stateless, et les microservices peuvent être groupés par type de charge au lieu de lancer des centaines de conteneurs séparés.

La performance vient de l'architecture, pas d'une seule astuce. Les données ne traversent pas de couches inutiles, les services possèdent leurs stores, Runtime parallélise les workflows et tâches cron, et les adaptateurs natifs sont utilisés là où HTTP ou une couche JS classique ajouterait trop d'overhead.

Une installation compacte peut fonctionner sur un petit serveur ou un single-board computer si la charge correspond à l'échelle de l'atelier. En grandissant, on peut séparer Runtime, microservices et groupes de storage pour utiliser plus de cœurs CPU, isoler les tâches lourdes et éviter qu'un goulot d'étranglement arrête tout le système.

La plateforme ne promet pas une performance infinie « out of the box ». Les goulots dépendent des équipements, du volume de fichiers, du nombre de commandes, des fournisseurs IA et des intégrations. L'architecture de Converged permet de commencer compact et de scaler uniquement les parties qui deviennent réellement chaudes.

Sécurité

Sécurité

Converged part du principe que les données de production ne doivent pas être jetées dans un espace commun. Commandes, fichiers clients, paramètres techniques, paiements, messages et télémétrie des équipements doivent être séparés par workspaces et zones de responsabilité.

Architecturalement, cela repose sur l'isolation des données. Les microservices possèdent leurs stores, et les workspaces peuvent avoir des répertoires, clés, fichiers et limites d'accès séparés. Cela simplifie l'export, la migration self-hosted, les sauvegardes et l'audit.

Les droits d'accès s'appliquent non seulement aux personnes, mais aussi aux agents IA. Si un modèle lance une action, lit des données ou appelle un workflow, cela doit se faire dans son profil de permissions. Les actions sont journalisées, ce qui permet de reconstruire qui ou quel agent a initié une étape, quelles données ont été touchées et comment le scénario s'est terminé.

Les déploiements self-hosted et private donnent au client le contrôle complet de l'infrastructure : réseau, secrets, clés API, sauvegardes et emplacement physique des données. Le mode cloud est plus simple opérationnellement, mais ne doit pas devenir un vendor lock-in : les données doivent rester portables, et les scénarios reproductibles dans une autre installation.

Licence

Licence

Converged est distribué sous **AGPL-3.0**. C'est une licence copyleft pour les logiciels réseau : si vous modifiez la plateforme et donnez accès à cette version via le réseau, les changements doivent être publiés selon les conditions de la licence.

Pour les utilisateurs, cela signifie que la version self-hosted peut être déployée sans acheter de licence pour le code lui-même. Cette voie convient aux entreprises qui ont besoin de contrôle, d'une installation locale, d'auditabilité ou d'expérimentation sur leur propre matériel. La responsabilité opérationnelle reste chez le propriétaire de l'installation : mises à jour, sauvegardes, monitoring, sécurité et disponibilité.

La livraison cloud n'est pas la vente d'un code fermé, mais un service autour d'une plateforme open-source. Le client paie pour le lancement rapide, le support, les mises à jour, les sauvegardes, le monitoring, l'infrastructure et une exploitation prévisible. Pour beaucoup d'ateliers, c'est moins coûteux que de construire une compétence DevOps en interne.

Cet équilibre est important pour la confiance : le cœur reste ouvert, la communauté peut vérifier et améliorer la plateforme, et le modèle commercial se construit autour de l'implémentation, du support et d'une livraison de valeur maîtrisée.

Communauté

Communauté

Converged est pensé comme une plateforme de production ouverte, pas comme une boîte SaaS fermée. Le cœur de base est disponible sous licence open-source, et des intégrations, microservices, micro-frontends, workflows et solutions applicatives peuvent grandir autour de lui.

C'est important dans l'industrie : les ateliers utilisent des équipements, matériaux, standards qualité et chaînes d'approvisionnement différents. Un système fermé atteint rapidement les limites d'un fournisseur unique. Une architecture ouverte permet d'ajouter des adaptateurs et scénarios adaptés aux conditions réelles.

Une extension peut être simple : un nouvel adaptateur d'équipement, une intégration de paiement, l'import d'un ancien site, un module UI séparé ou un workflow pour une industrie précise. Si l'extension est utile à d'autres participants, elle peut entrer dans le catalogue de solutions ou rester une implémentation privée.

Ouvert ne signifie pas sans contrôle. Les extensions doivent passer une revue technique, respecter les limites architecturales et ne pas recevoir plus d'accès aux données que nécessaire. La confiance dans l'écosystème repose sur le code source, la reproductibilité et un modèle clair de permissions.

Code source

Code source

Le projet est développé ouvertement. Le code source, l'architecture actuelle et les matériaux de travail sont disponibles dans le dépôt :

<https://github.com/solenopsys/converged>

Le dépôt n'est pas utile uniquement aux développeurs. Il permet de comprendre l'organisation des microservices, de Runtime, des micro-frontends, de la génération de contrats, des profils de déploiement et des adaptateurs matériels. Pour les entreprises qui envisagent un déploiement self-hosted ou private, c'est une partie importante de l'évaluation : la plateforme n'est pas une boîte fermée.